

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

GABRIEL HENRIQUE ROCHA MELO

**RELATÓRIO DE PROGRESSO NA UNIDADE 2 DO PROJETO DO COMPONENTE
CURRICULAR INTRODUÇÃO ÀS TÉCNICAS DE PROGRAMAÇÃO**

NATAL-RN
2025

INTRODUÇÃO

O projeto, de nome: Calculadora Científica, tem o objetivo de proporcionar ao usuário a capacidade de realizar operações matemáticas mais complexas e específicas, além das operações básicas, que uma calculadora convencional pode realizar, como cálculo estatístico, trigonométrico e matricial. O tema do projeto foi escolhido como uma forma tanto de aplicar os conhecimentos e práticas aprendidos no decorrer da segunda unidade do componente curricular, como alocação dinâmica de memória e laços aninhados, e portanto um objeto avaliativo para esse aprendizado, quanto como um projeto de âmbito pessoal que possivelmente pode evoluir para algo maior após a conclusão do curso de ITP, visto que é notável o uso de conteúdos externos à segunda unidade como modularização, definição de estrutura de dados e recursão simples. Existem 29 funções implementadas para o funcionamento da calculadora, desde a função para o processamento das opções do menu principal, até funções extras que serão utilizadas nas funções principais das operações e do histórico, sendo 11 funções principais para as diversas operações matemáticas e as outras 18 como auxiliares dessas 11 funções, como as funções de alocação e as funções para cálculo matricial.

METODOLOGIA

- **Ferramentas utilizadas:** Foram utilizadas como ferramentas para a elaboração do projeto o compilador GCC(GNU Compiler Collection), o editor de código-fonte gratuito Visual Studio Code(VS Code) e o GitHub Desktop para importações e alterações necessárias de arquivos pertinentes ao projeto.
- **Abordagem de desenvolvimento:** Para o desenvolvimento do projeto foram utilizados em grande extensão os conceitos adquiridos na unidade 1, unidade 2 e outros conhecimentos externos às essas unidades. O laço de repetição mais importante no desenvolvimento é o do-while utilizado na função main para chamadas constantes de seleções de opções do menu principal a cada nova solução das operações para não apenas permitir o usuário explorar as diversas outras funções da calculadora, como também é uma ferramenta importante na atualização constante do tamanho do histórico. Laços de repetição while e do-while foram utilizados dentro das funções principais para verificação das opções selecionadas dentro dos menus de operações e, caso necessário, verificação de outros valores como as dimensões de uma matriz. Laços de repetição unidimensionais(for simples) foram utilizados basicamente para verificação do vetor utilizado em questão e, como no caso das operações estatísticas e sua verificação do número moda de uma sequência de números do tipo double, para o auxílio da operação principal a qual a função auxiliar se destina, evidenciado também pela função de ordenação cujo método utilizado foi o insertion sort devido sua eficiência na ordenação dos valores inteiros usados ser melhor que Bubble sort e o Selection Sort. O tipo de laço de repetição mais utilizado foi o bidimensional para as funções envolvendo matrizes e para a comparação de valores de uma sequência numérica na função estatísticas de moda. Ponteiros e alocação de memória foram utilizados em praticamente qualquer vetor que precisaria de um tamanho de alocação variável, como a função histórico que, atendido certas condições, era necessário aumentar o tamanho da memória alocada, ou matrizes que podem ter tamanhos diversos(5x7, 8x4, 7x2, 3x3). Para a função histórico a solução encontrada foi transformar cada resultado encontrado em uma string que seria então adicionada a outra string final, sendo essa última a string histórico, para isso foi utilizado a função sprintf e strcat para pleno funcionamento das funções. Devido ao número exorbitante de funções, foi necessário a modularização, criando assim 6 arquivos .c e 6 arquivos .h para diferentes bibliotecas de criação autoral. Todas as funções das operações implementadas são funções que retorna um ponteiro para uma string, retornando a string do histórico que teve o resultado de uma operação armazenada dentro dela.
- **Organização do código:** Devido a modularização, o código foi dividido em 6 arquivos .c: o arquivo main2.c(projeto da unidade 2) e outros 5 arquivos .c contendo as funções pertinentes projeto como a função de fatoração em primos e as funções principais para as operações implementadas. Foi criado também 6 arquivos .h(bibliotecas autorais) com 5 delas contendo os cabeçalhos das funções implementadas nos arquivos .c e sexta sendo para uma estrutura de dados criada para ser utilizado nas dimensões das matrizes.

ANÁLISE DE CÓDIGO

- **Strings:**

- **Como foram utilizadas no projeto?:** Foram utilizados apenas para implementação do histórico, transformando todos os resultados em strings que seriam transferidos para a string final do histórico, com o auxílio das funções `strcat` e `sprintf`.
- **Quais funções de manipulação foram implementadas?:** `Strcat` e `sprintf`
- **Exemplos de uso:** `sprintf(entrada, "%.2lf x %.2lf = %.2lf\n", n1, n2, *resultado); hist = adicionarhistorico(hist, entrada, espacousado, tamanho);`. Aqui é onde o resultado de uma operação é armazenado em uma string de tamanho 5000 e posteriormente essa string entrada é armazenada pela função `adicionar histórico`.

- **Estrutura de Repetição Aninhada:**

- **Onde foram aplicadas?:** Foram aplicadas em todas as funções envolvendo matrizes, na função de moda das funções estatísticas e na função de fatoração em primos.
- **Qual a complexidade dos algoritmos?:** A complexidade dos algoritmos de matrizes está na faixa do médio-difícil, já que necessita de uma mudança no paradigma de visualização matricial em linguagem c devido ao alocamento de memória, além do pensamento matemático mais complexo e uma visualização mais consolidada de como manipular strings em uma repetição aninhada. O algoritmo do número moda é algo do nível simples-médio devido ao fato de que é apenas usado para uma comparação entre elementos de um vetor que foi ordenado. O algoritmo de fatoração em primos é de complexidade média-difícil já que não envolve apenas o raciocínio matemático da fatoração em primos, mas também a manipulação de strings e dois vetores (um para os fatores primos e outro para suas potências) para seu pleno funcionamento e posteriormente ser adicionado ao histórico.
- **Casos de usos:** Algoritmo implementado para adicionar uma matriz resultado ao histórico, transformando tudo em uma string (função: `void matriz_historico(char *entrada, double *matrizresultado, dimensao tamanho)`).

- **Matrizes:**

- **Como foram implementadas?:** Foram implementadas em todas as operações e funções auxiliares envolvendo matrizes e apenas nelas, como nas funções de leitura e impressão que utilizaram de laços de repetição aninhados (um `for i` e um `for j`), já que esse é o algoritmo padrão ao lidar com matrizes.
- **Quais operações são suportadas?:** Soma, subtração e multiplicação matricial.

- **Estratégias de percorrimento:** Para as funções de soma, subtração e multiplicação matricial foram criados uma única matriz resultado que receberia o resultado das operações matriciais. Na soma e subtração, de lógica semelhante, o primeiro elemento da matriz resultado receberia a soma/subtração entre o primeiro elemento da matriz 1 e da matriz 2. Na multiplicação matricial foi necessário uma repetição aninhada em 3 laços de repetição(for i, for j e for k), devido ao fato da lógica envolvendo o recebimento dos valores da matriz resultado necessitar de mais uma variação interna para que os valores da multiplicação e soma envolvendo a multiplicação matricial sejam armazenadas nos índices certos. Na transformação do resultado das operações matriciais em uma string formatada na forma de: **matriz 1 “operação” matriz 2 = matriz resultado**, que seria então levada para o histórico, foram necessários vários laços de repetição para percorrer linha por linha de cada vez de cada matriz, e também para verificar a quantidade de espaços que cada coluna de cada matriz deveria ocupar, transformando esses espaços desnecessários em uma sequência de espaços vazios(“ ”) através de um for, com tudo isso dentro de um único laço de repetição que percorre as linhas de cada matriz.

- **Ponteiros:**

- **Como foram utilizados?:** Foram utilizados sempre que um vetor exigia um tamanho variável e principalmente para a função do histórico que, pelo algoritmo implementado, precisava ter seu tamanho e conteúdo alterados. Por isso os parâmetros necessários para essa operação fazem parte do cabeçalho de cada função principal, já que esses valores serão pertinentes para adicionar novas operações ao histórico e aumentar seu tamanho caso precise.
- **Passagem por referência vs. valor:** A passagem por referência foi de extrema importância para o desenvolvimento do projeto, pois graças a esse método, ao contrário da passagem por valor que apenas copia o valor de uma variável sem alterar seu original, me permitiu fazer retornos indiretos(retornar mais de um valor em uma função) que me permitiram alterar o funcionamento da função histórico, alterando seu tamanho e espaço de memória alocado, alterando o valor original da variável.
- **Exemplos práticos:** processar_opcao(opcao, historico, &espacousado, &tamanho_inicial_historico). Essa função passa por referencia(endereço na memória) as variáveis tamanho_inicial_historico e espacousado, previamente definidos com valores iniciais, que, através do endereço de memória, serão alteradas posteriormente conforme a necessidade.

- **Alocação Dinâmica:**

- **Onde foi necessária?:** Foi necessária em todas as operações que envolvem vetores(vetores simples, matrizes e strings).
- **Como é gerenciada a memória?:** Ela é gerenciada utilizando da função de alocação malloc e posteriormente a memória utilizada é liberada pelo comando

free.

- **Tratamento de falhas de alocação:** Para toda função de alocação de memória é retornado o valor NULL no caso da alocação de memória falhar, seja por espaço insuficiente na memória ram ou foi usado um valor inválido para o tamanho. Isso garante que ao retornar o vetor alocado, não retorne um endereço de memória inválido, já que ao tentar alterar ou acessar o vetor alocado de qualquer forma, estará tentando acessar um endereço de memória inválido e, portanto, o código estará tentando acessar uma área da memória que ele não tem acesso. E caso o resultado retornado seja NULL, é utilizado, como na função de matrizes, uma recursão simples para solicitação de novos valores.
- **Estratégia para evitar memory leaks:** É utilizado o comando free em todos os ponteiros alocados dinamicamente que não sejam mais pertinentes ao cálculo das operações em questão, após seus resultados serem transformados em strings para o histórico. Também, quando o usuário digita zero no menu principal(sair da calculadora), é liberado então o espaço de memória da string do histórico.

DIFICULDADES E SOLUÇÕES

Os principais desafios envolveram principalmente na parte de alocação de memória, quando utilizar o malloc e quando(em que linha) no código utilizar o comando free, como utilizar corretamente a modularização e recursão e um certo grau de dificuldade em ponteiros e passagem por referência. Fora esses desafios o resto foi muito mais tranquilo, já que era apenas uma aplicação mais aprofundada dos conteúdos da unidade 1. Esses desafios foram superados através de constante pesquisa e estudo, utilizando vídeo aulas(gravadas durante as reuniões assíncronas ou outros vídeos do youtube), conteúdos disponíveis publicamente de outras instituições de ensino, e entre outros, que me ajudaram a elucidar em grande parte as dúvidas adquiridas no decorrer do projeto. Com toda certeza os aprendizados mais importantes foram as habilidades adquiridas nos conteúdos de alocação de memória e tudo o que envolve ponteiros simples e passagem por referência. Com ambos é notável a importância quando se está trabalhando na construção de estruturas de dados complexas ou de tamanho considerável, pois é nos permitido através dessas duas ferramentas manipular a memória ram de forma flexível, segura e eficiente, para a quantidade necessária, alterando-a conforme a necessidade, permitindo-nos situações de retorno indireto, por exemplo, em que uma única função pode retornar mais que 1 valor.

CONCLUSÃO

Por fim, é notável que todo o conhecimento adquirido no decorrer da unidade 2 e implementado ao longo do desenvolvimento do projeto, serão de grande utilidade não só para futuros projetos acadêmicos, mas os projetos de âmbito pessoal também, já que as habilidades adquiridas, principalmente de ponteiros, alocação e manipulação dinâmica de memória será de grande utilidade no mundo profissional e acadêmico. É perceptível também uma melhoria considerável dos conteúdos vistos na unidade 1, como, por exemplo, uma visualização mais intuitiva e menos rígida de estruturas condicionais(switch-case ao invés de if-else) e vetores(strings, laço aninhado) e declarações de variáveis(passagem por referência). Ainda

assim, é possível melhorar, desde que sejam estudadas novas ferramentas de implementação e lógicas mais avançadas de matemática.

PERGUNTAS OBRIGATÓRIAS

- **Gerais:**

- **Quais conceitos da unidade 2 foram utilizados?:** Todos.
- **Como a organização do código facilita a manutenção?:** Como foi feito a modularização, qualquer alteração no código é feita apenas indo na biblioteca respectiva e fazendo as alterações necessárias nos arquivos .h e .c, sem alteração do código principal(main2.c). Também, como dentro das funções estão todas as necessidades que ela precisa, como a verificação de opções dos menus secundários e os menus secundários, basta apenas alterar uma única função e sua biblioteca auxiliar, caso necessário, dispensando alteração em múltiplas funções extras sem necessidade.
- **Quais foram os principais desafios técnicos enfrentados?:** Utilização de ponteiros, alocação dinâmica de memória, utilização correta de modularização e recursão e lógica por trás do histórico ilimitado.

- **Específicas da unidade 2:**

- **Como foram implementadas as estruturas de dados complexas (matrizes)?:** A maioria das matrizes foram implementadas com repetição aninhada dupla(for i e for j), mas a multiplicação de matriz e a função que transforma o resultado matricial em uma string utilizaram repetição aninhada tripla(for i, for j e for k).
- **Qual a estratégia para gerenciamento de memória?:** Uso do comando malloc para alocação dinâmica de memória e free para a liberação da memória(evita memory leak), retornando NULL caso ocorra erro na alocação(memória indisponível ou outra coisa).
- **Como você garante que não há vazamentos de memória?:** Foi utilizado free sempre que o vetor, matriz ou string não é mais pertinente para a execução da calculadora.
- **Quais vantagens a alocação dinâmica trouxe para seu projeto?:** Manipulação da quantidade de memória ram de forma flexível, segura e eficiente, para a quantidade necessária, alterando-a conforme a necessidade, como nas funções do histórico que, caso necessário, devem ter seu tamanho expandido e, por consequência, seu espaço na memória necessário para armazenamento.
- **Como os ponteiros foram utilizados para melhorar a eficiência?:** Passagem por referência, permitindo retorno indireto e alteração dinâmica dos valores originais das variáveis utilizadas.